

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC MỞ THÀNH PHỐ HỒ CHÍ MINH

Lê Gia Bảo

XÂY DỰNG HỆ THỐNG TRỢ LÝ PHÁP LÝ AI CHO
DOANH NGHIỆP NHỎ VÀ VỪA

ĐỒ ÁN NGÀNH

NGÀNH CÔNG NGHỆ THÔNG TIN

TP. HỒ CHÍ MINH, 2026

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC MỞ THÀNH PHỐ HỒ CHÍ MINH

Lê Gia Bảo

XÂY DỰNG HỆ THỐNG TRỢ LÝ PHÁP LÝ AI CHO
DOANH NGHIỆP NHỎ VÀ VỪA

Mã số sinh viên: <MSSV>

ĐỒ ÁN NGÀNH

NGÀNH CÔNG NGHỆ THÔNG TIN

Giảng viên hướng dẫn: ThS. Dương Hữu Thành

TP. HỒ CHÍ MINH, 2026

LỜI CẢM ƠN

Em xin cảm ơn ThS. Dương Hữu Thành đã nhận hướng dẫn đồ án này. Những góp ý của thầy về cách tách phần truy hồi khỏi tầng HTTP, và về việc không để hệ thống “chết cả cụm” chỉ vì thiếu khóa API, đã đổi hướng hiện thực khá nhiều so với bản phác đầu tiên của em.

Em cũng cảm ơn các thầy cô Khoa Công nghệ Thông tin, Trường Đại học Mở Thành phố Hồ Chí Minh. Phần lớn kiến thức em dùng khi viết lược đồ CSDL, thiết kế use case và đóng gói Docker đều xuất phát từ các học phần đã học, chứ không phải tự nghĩ ra trong vài tuần làm đồ án.

Trong quá trình làm, em còn dựa vào kho văn bản pháp luật đã công bố và vào pipeline truy hồi từng dùng cho một cuộc thi hỏi đáp pháp lý tiếng Việt. Đồ án không nộp lại bài thi đó; em chỉ lấy phần lõi truy hồi rồi viết thành ứng dụng có đăng nhập, lịch sử hội thoại và vài nghiệp vụ dành cho doanh nghiệp nhỏ.

Mọi chỗ còn vụng, còn thiếu số liệu hay còn nói chưa chặt, là trách nhiệm của em.

NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN

TP. Hồ Chí Minh, ngày ... tháng ... năm 2026

Giảng viên hướng dẫn

ThS. Dương Hữu Thành

TÓM TẮT ĐỒ ÁN NGÀNH

Đồ án này làm một việc khá cụ thể: viết một trang web để người làm trong doanh nghiệp nhỏ — chủ cửa hàng, kế toán, nhân sự — hỏi được câu hỏi pháp lý bằng tiếng Việt thường ngày, và nhận câu trả lời kèm đúng Điều luật để tự mở ra đối chiếu. Đồ án không nhằm thay thế luật sư. Mục tiêu hẹp hơn: giảm việc đoán mò, và giảm việc copy câu trả lời từ chatbot không nêu nguồn.

Phần hỏi đáp dùng kiểu truy hồi-rời-sinh (RAG). Corpus gồm 38 văn bản, 3.844 Điều, lấy từ các luật, nghị định, thông tư gắn với thuế, lao động, bảo hiểm, kế toán, hợp đồng và hỗ trợ DNNVV. Mỗi Điều là một đơn vị lưu trữ. Khi có câu hỏi, hệ thống tìm song song theo nghĩa (vector) và theo từ (BM25), gộp bằng xếp hạng bằng RRF, lọc bớt Điều nhiều bằng mô hình, rồi mới gọi LLM. Bước cuối đối chiếu các số Điều xuất hiện trong câu trả lời với tập vừa truy hồi; số nào không có trong tập đó thì bị gỡ khỏi danh sách căn cứ hiện cho người dùng.

Ngoài hỏi đáp, đồ án còn ba mảng. Tra cứu văn bản cho phép đi theo cây Chương rồi tới Điều, tìm cả câu không dấu mà đoạn trích vẫn còn dấu. Soát xét hợp đồng nhận file PDF hoặc DOCX, cắt theo điều khoản rồi chấm mức rủi ro. Lịch tuân thủ sinh các mốc khai thuế GTGT, quyết toán TNDN, đóng BHXH, báo cáo lao động và nộp BCTC theo hồ sơ doanh nghiệp đã khai, mỗi mốc kèm `legal_refs`.

Hệ thống viết bằng React (phía trình duyệt) và FastAPI (phía máy chủ), dữ liệu nằm trên PostgreSQL, vector nằm trên Chroma, đóng gói bằng Docker Compose. Bộ kiểm thử tự động hiện có 77 test. Khi chưa gắn khóa dịch vụ mô hình, đăng nhập và tra cứu văn bản vẫn chạy; chỉ đường hỏi đáp báo chưa sẵn sàng. Lựa chọn này xuất phát từ thực tế lúc em cài: thiếu khóa thì không nên kéo theo cả trang web sập.

Báo cáo gồm năm chương: giới thiệu, cơ sở lý thuyết, phân tích–thiết kế–hiện thực, thử nghiệm–đánh giá, kết luận và hướng phát triển.

ABSTRACT

This project is a web application that lets people in small Vietnamese firms ask everyday legal questions and get answers that point back to real articles in the statute book. It is not a substitute for a lawyer. The narrower aim is to stop people from guessing, and to stop them from copying unsourced chatbot text into a tax filing or a labour contract.

Question answering follows a retrieve-then-generate pattern. The corpus holds 38 legal texts and 3.844 articles covering tax, labour, social insurance, accounting, contracts and SME support. Retrieval runs BM25 and a vector index in parallel, fuses the two ranked lists, drops weak context, then calls a language model. Article numbers that the model invents but that were never retrieved are stripped from the citation list shown to the user.

The same codebase also offers a chapter-to-article browser with diacritic-tolerant search, a contract reviewer that scores clauses from a PDF or DOCX upload, and a compliance calendar generated from the company's size and VAT filing period.

The stack is React and FastAPI on PostgreSQL and Chroma. It is shipped with Docker Compose and covered by 77 automated tests. If the model API key is missing, login and the law browser still work; only the chat path returns a clear error. That behaviour was a deliberate choice after the first deployments failed as a whole unit whenever embeddings could not be built.

Mục lục

DANH MỤC TỪ VIẾT TẮT	VIII
DANH MỤC HÌNH VẼ	IX
DANH MỤC BẢNG	X
1 Giới thiệu đề tài	1
1.1 Tổng quan tình hình liên quan đến đề tài	1
1.2 Lý do lựa chọn đề tài	1
1.2.1 Về phía người dùng	1
1.2.2 Về phía kỹ thuật	2
1.3 Mục tiêu đề tài	2
1.3.1 Mục tiêu tổng quát	2
1.3.2 Mục tiêu cụ thể	2
1.4 Cách tiếp cận và phương pháp	3
1.4.1 Open-book QA thay vì close-book QA	3
1.4.2 Hybrid retrieval	3
1.4.3 Phát triển sản phẩm theo vòng ngắn	3
1.5 Đối tượng và phạm vi	3
1.5.1 Đối tượng	3
1.5.2 Phạm vi chức năng	3
1.5.3 Phạm vi dữ liệu và kỹ thuật	3
1.6 So sánh nhanh với công cụ sẵn có	4
1.7 Phần em nhận là đóng góp	4
1.8 Bố cục báo cáo	4
2 Cơ sở lý thuyết	5
2.1 Xử lý ngôn ngữ tự nhiên trong bài toán hỏi đáp	5
2.2 Cấu trúc văn bản quy phạm pháp luật Việt Nam	5
2.2.1 Luật, nghị định, thông tư không nằm trên một trang	5
2.2.2 Điều là đơn vị cắt dữ liệu	5
2.2.3 Viện dẫn chéo	6

2.3	Truy hồi thưa: BM25	6
2.4	Truy hồi dày: embedding và cosine	6
2.5	Gộp hạng: Reciprocal Rank Fusion	6
2.6	RAG, HyDE và rủi ro bịa nguồn	7
2.6.1	RAG	7
2.6.2	HyDE	7
2.7	Chỉ số đánh giá retrieval: vì sao F2	7
2.8	Word embedding và đặc thù tiếng Việt (nhìn từ chỗ cài)	8
2.9	Prompt engineering trong đồ án	8
2.10	Full-text search tiếng Việt trên PostgreSQL	8
2.11	Công nghệ hiện thực và lý do chọn	9
2.12	Bảng tổng kết: vai trò – thách thức – hướng xử lý	9
2.13	Việc người khác đã làm, em lấy gì	9
3	Phân tích, thiết kế và hiện thực	12
3.1	Hệ thống nhìn từ phía người bấm	12
3.2	Yêu cầu	12
3.2.1	Chức năng	12
3.2.2	Phi chức năng	13
3.3	Kiến trúc ba lớp	13
3.4	Use case	14
3.4.1	Đặc tả bổ sung UC-05 và UC-09	14
3.5	Bảy bước hỏi đáp	14
3.6	Corpus	17
3.7	Mô hình dữ liệu	18
3.7.1	Ba chỗ em làm sai rồi sửa	18
3.8	Tìm chữ trên tiếng Việt	19
3.9	Luồng thời gian thực và lưu hội thoại	20
3.10	Hợp đồng	20
3.11	Lịch tuân thủ	21
3.12	Đăng nhập, phiên làm việc, bảo mật	21
3.13	Giao diện	22
3.14	Nhúng vector, cache BM25, và chuyện file .env	22
3.15	Đóng gói ba dịch vụ	23
3.16	Tóm tắt hiện thực theo module	23
4	Thử nghiệm và đánh giá	25
4.1	Môi trường thử nghiệm	25
4.2	Kiểm thử tự động	25
4.2.1	Tổ chức bộ test	25

4.2.2	Các case viết vì từng vớ tay	25
4.3	Chạy tay trên cụm Docker	26
4.4	Kịch bản hỏi đáp chạy tay	26
4.4.1	Câu trong phạm vi corpus	26
4.4.2	Câu không phải pháp luật	26
4.4.3	Câu ngoài corpus	27
4.5	Đánh giá module tra cứu và lịch tuân thủ	27
4.6	Đánh giá giao diện (định tính)	27
4.7	Biểu đồ tóm tắt bộ kiểm thử	29
4.8	Vì sao không có cột F2 cuộc thi trong báo cáo	29
4.9	Đối chiếu mục tiêu Chương 1	30
5	Kết luận và hướng phát triển	31
5.1	Những gì đề án thực sự làm được	31
5.2	Những gì cố tình không làm, và những gì còn yếu	31
5.3	Hướng phát triển	31
5.4	Kết luận	32
	TÀI LIỆU THAM KHẢO	33
	PHỤ LỤC	34

DANH MỤC TỪ VIẾT TẮT

Viết tắt	Đầy đủ		Dùng trong đề án để chỉ
API	Application Interface	Programming	Cổng gọi chức năng của máy chủ
BHXX	Bảo hiểm xã hội		Nghĩa vụ đóng bảo hiểm của doanh nghiệp
BM25	Best Matching 25		Cách chấm điểm khớp từ khi tìm Điều luật
BCTC	Báo cáo tài chính		Báo cáo năm doanh nghiệp phải nộp
CSDL	Cơ sở dữ liệu		PostgreSQL trong đề án
DNNVV	Doanh nghiệp nhỏ và vừa		Nhóm người dùng em nhắm tới
FTS	Full-text search		Tìm chữ trong corpus, khác với tìm theo nghĩa
GTGT	Giá trị gia tăng		Thuế khai theo tháng hoặc theo quý
HyDE	Hypothetical Embeddings	Document	Sinh một đoạn trả lời giả để đi tìm vector
JWT	JSON Web Token		Token đăng nhập, gồm access và refresh
LLM	Large Language Model		Mô hình sinh câu trả lời
ORM	Object-Relational Mapping		SQLAlchemy ánh xạ bảng thành đối tượng
RAG	Retrieval-Augmented Generation		Sinh câu trả lời sau khi đã tìm văn bản
RRF	Reciprocal Rank Fusion		Cách gộp hai bảng xếp hạng BM25 và vector
SPA	Single Page Application		Giao diện React một trang
SSE	Server-Sent Events		Đẩy token và tiến độ từ máy chủ xuống trình duyệt
TNCN	Thu nhập cá nhân		Thuế khấu trừ tiền lương
TNDN	Thu nhập doanh nghiệp		Thuế trên lợi nhuận
VBQPPL	Văn bản quy phạm pháp luật		Luật, nghị định, thông tư trong corpus

Danh sách hình vẽ

3.1	Ba lớp triển khai: trình duyệt → nginx → FastAPI; PostgreSQL, Chroma và Gemini nằm phía sau API.	13
3.2	Sơ đồ use case. Chủ / kế toán / nhân sự dùng bốn nghiệp vụ; quản trị thêm corpus và lab.	15
3.3	Bảy nút LangGraph: nhánh SKIP thoát sớm; rerank tắt trên sản phẩm Gemini. .	17
3.4	Kiến trúc ba lớp: trình bày, ứng dụng, dữ liệu.	18
3.5	Lược đồ quan hệ chính. Hội thoại thuộc user; hợp đồng và lịch thuộc organization.	18
3.6	Luồng hỏi đáp: ghi câu hỏi trước, stream SSE, ghi câu trả lời ở session CSDL riêng.	20
3.7	Luồng soát xét hợp đồng: trả 202 ngay, xử lý nền, UI poll trạng thái.	20
3.8	Máy trạng thái tài liệu: pending → ready → processing → done/failed.	20
4.1	Bố cục trang hỏi đáp: cột hội thoại, vết pipeline, câu trả lời và citation.	27
4.2	Bố cục trang tra cứu: cây Chương—Điều và tìm không dấu.	28
4.3	Bố cục trang hợp đồng: upload và bảng rủi ro.	28
4.4	Bố cục trang lịch tuân thủ: ô đếm và nhóm theo tháng.	29
4.5	Phân bố số test tự động đạt trên máy làm đồ án.	29

Danh sách bảng

1.1	Ba kiểu công cụ em đối chiếu trước khi chốt hướng.	4
2.1	Công nghệ gắn vào sản phẩm.	9
2.2	Tổng kết khối truy hồi hybrid.	10
2.3	Tổng kết khối sinh câu và hậu kiểm trích dẫn.	10
2.4	Tổng kết khối nghiệp vụ DNNVV (hợp đồng + lịch).	10
3.1	Yêu cầu chức năng em chốt trước khi viết API.	12
3.2	Ngưỡng phi chức năng em tự đặt khi viết code.	13
3.3	Tác nhân.	14
3.4	Danh sách use case.	14
3.5	Đặc tả UC-03 — Hỏi đáp.	15
3.6	Đặc tả UC-07 — Soát xét hợp đồng.	16
3.7	Đặc tả UC-05 — Tra cứu và đọc văn bản.	16
3.8	Đặc tả UC-09 — Lịch tuân thủ.	16
3.9	Đếm nhanh theo loại và theo lĩnh vực (một văn bản vào đúng một lĩnh vực). . .	18
3.10	Bảng. Tên cột chỉ liệt kê những cột em từng vấp khi cài.	19
3.11	Mười hai rule em cài, và điều kiện hồ sơ để rule được sinh.	21
3.12	Ánh xạ module code ↔ nghiệp vụ.	23
4.1	Cấu hình máy và cụm khi chạy thử.	25
4.2	Kết quả smoke test trên cụm Docker em ghi lại.	26
4.3	Đối chiếu hồ sơ doanh nghiệp và task được sinh.	27
4.4	Đối chiếu bảy mục tiêu cụ thể với bằng chứng.	30

Chương 1. GIỚI THIỆU ĐỀ TÀI

1.1 Tổng quan tình hình liên quan đến đề tài

Ở Việt Nam, doanh nghiệp nhỏ và vừa (DNNVV) phải tự đối mặt với một khối lượng văn bản quy phạm khá dày: thuế, lao động, bảo hiểm, kế toán, hợp đồng thương mại, sở hữu trí tuệ ở mức đăng ký nhãn hiệu. Cổng thông tin văn bản quy phạm pháp luật nhà nước mở và đáng tin, nhưng cách vào cổng đó giả định người hỏi đã biết số hiệu hoặc ít nhất biết thuật ngữ pháp lý đúng. Thực tế kế toán hoặc chủ cửa hàng thường gõ câu nói thường ngày: “thử việc được bao lâu”, “chậm đóng BHXH phạt thế nào”, “quý này nộp GTGT ngày nào”. Kết quả tìm theo từ khóa lúc thì quá rộng, lúc thì trượt vì luật viết “đơn phương chấm dứt hợp đồng lao động” chứ không viết “sa thải”.

Song song đó, các chatbot ngôn ngữ lớn trả lời tiếng Việt rất trôi. Em thử nhiều câu hỏi thật của một công ty thương mại khoảng mười hai người. Máy trả lời tự tin, đôi khi còn nêu số Điều. Khi đối chiếu văn bản gốc, có lần số Điều không tồn tại, có lần nội dung bị trộn từ nghị định khác. Với nghĩa vụ có phạt hành chính, một câu sai mà nghe chắc chắn còn nguy hiểm hơn là không có câu trả lời.

Trên hướng kỹ thuật, kiến trúc truy hồi-rời-sinh (RAG) [3] đã được dùng rộng rãi để buộc mô hình nhìn vào ngữ cảnh đã tìm trước khi viết. Các công trình về dense retrieval [4], BM25, Reciprocal Rank Fusion [6] và HyDE [7] cung cấp đủ “gạch” để ghép một pipeline hỏi đáp trên corpus hẹp. Ở môi trường tiếng Việt trong trường, bài chatbot tuyển sinh dùng RAG [2] cho thấy hướng này chạy được khi kho tri thức có biên rõ. Em không làm lại thí nghiệm của các bài đó; em lấy ý rồi áp vào kho luật DNNVV có cấu trúc Điều–Chương.

Có một nhánh riêng: pipeline truy hồi pháp lý từng dùng trong cuộc thi hỏi đáp tiếng Việt (mô hình mở, ràng buộc tham số). Đề án **không nộp lại bài thi**. Em kế thừa lõi cắt theo Điều và hybrid BM25–vector, rồi viết thành ứng dụng web có đăng nhập, lịch sử hội thoại, tra cứu, soát hợp đồng và lịch tuân thủ. Sản phẩm mặc định gọi Gemini qua lớp tương thích OpenAI — khác hẳn ràng buộc mô hình mở của cuộc thi — nên số F2 trên leaderboard không được đem gắn vào kết quả đề án.

1.2 Lý do lựa chọn đề tài

1.2.1 Về phía người dùng

Doanh nghiệp nhỏ ít khi thuê được pháp chế riêng, nhưng nghĩa vụ không thu hẹp theo quy mô. Khai thuế GTGT, tạm nộp và quyết toán TNDN, đóng BHXH, báo cáo tình hình sử dụng lao động, nộp báo cáo tài chính năm — các mốc này lặp lại, có hạn chót, và chậm là có chế tài. Một công cụ nhắc việc kèm chỗ dẫn luật, theo em, hữu ích hơn một chatbot chỉ khuyên chung “doanh

nghiệp cần tuân thủ pháp luật”.

Em cố tình không kéo đề tài sang tổ tụng, hình sự hay đất đai chuyên sâu. Làm hẹp để làm xong và kiểm chứng được. Phạm vi DNNVV đã đủ lớn cho một đề án ngành.

1.2.2 Về phía kỹ thuật

Ba chỗ không sao chép được từ hướng dẫn “cài chatbot có sẵn”:

1. **Lịch từ vựng.** Người dùng nói “sa thải”; luật viết “đơn phương chấm dứt”. Chỉ BM25 thì trượt; chỉ vector thì đôi khi bắt nhầm Điều kề nhau. Phải hybrid rồi gộp hạng.
2. **Tiếng Việt trên PostgreSQL.** Không có từ điển FTS tiếng Việt chuẩn. Người dùng hay gõ không dấu. Nếu bỏ dấu rồi vừa tìm vừa hiện, đoạn trích mất dấu — không đọc được. Phải tách tìm (bản không dấu) và hiện (bản còn dấu, bôi đậm phía client).
3. **Bịa số Điều.** Nếu chỉ tin đầu ra mô hình, đề án không khác chatbot kia. Em thêm hậu kiểm: số Điều nêu trong câu trả lời phải nằm trong tập vừa truy hồi; không thì gỡ khỏi danh sách căn cứ hiện ra.

1.3 Mục tiêu đề tài

1.3.1 Mục tiêu tổng quát

Xây dựng một ứng dụng web chạy được, nhiều người dùng, nhiều doanh nghiệp, trong đó phần pháp lý đưa ra đều lần ra được văn bản gốc trong một corpus đã chốt.

1.3.2 Mục tiêu cụ thể

1. Gom và chuẩn hóa corpus DNNVV: 38 văn bản, 3.844 Điều, 452 chương; cắt theo Điều; giữ metadata và viện dẫn chéo.
2. Cài pipeline hỏi đáp bảy bước: phân tích ý định, chuẩn bị truy vấn (HyDE), truy hồi lai, lọc LLM, sinh câu, hậu kiểm trích dẫn; stream SSE.
3. Trang tra cứu: cây Chương–Điều, tìm không dấu, nút hỏi tiếp về Điều đang đọc.
4. Soát xét hợp đồng PDF/DOCX/TXT, chạy nền, poll trạng thái, bảng rủi ro.
5. Lịch tuân thủ 12 loại nghĩa vụ, sinh theo hồ sơ công ty, kèm `legal_refs`.
6. Đăng nhập JWT (access + refresh), bốn vai trò, cách ly dữ liệu.
7. Docker Compose ba dịch vụ và bộ test tự động 77 test.

Mục tiêu em **cố ý không nhận**: đạt điểm F2 cao nhất trên bảng xếp hạng cuộc thi với đúng mô hình mở. Lý do đã nêu ở mục tổng quan.

1.4 Cách tiếp cận và phương pháp

1.4.1 Open-book QA thay vì close-book QA

Close-book QA nhờ mô hình “nhớ” kiến thức trong tham số. Với luật Việt Nam hay sửa và hay bịa số Điều, close-book không phù hợp. Em chọn open-book: mọi kết luận pháp lý phải dựa trên đoạn văn đã truy hồi từ corpus [3]. Nếu không tìm ra Điều liên quan, hệ thống phải nói thiếu căn cứ, không được viết đoạn khuyên chung giả danh luật.

1.4.2 Hybrid retrieval

Dense retrieval bắt nghĩa; BM25 bắt mặt chữ và số hiệu. Reciprocal Rank Fusion gộp hai bảng xếp hạng mà không cần chuẩn hóa điểm về cùng thang [6]. Trong cấu hình đồ án, trọng số $\text{dense:BM25} = 3:1$, $k_{\text{RRF}} = 60$, lấy top $k = 8$ Điều đưa vào các bước sau.

1.4.3 Phát triển sản phẩm theo vòng ngắn

Em không viết hết tài liệu thiết kế rồi mới mở editor. Thứ tự thực tế: chốt corpus $\rightarrow$ Postgres và bảng tri thức $\rightarrow$ tài khoản và hội thoại $\rightarrow$ tra cứu $\rightarrow$ hợp đồng $\rightarrow$ lịch tuân thủ. Mỗi mảng xong API thì làm giao diện ngay, vì nhiều lỗi chỉ lộ khi bấm tay (định tuyến số hiệu có dấu /, thứ tự tin nhắn cùng `created_at`, FormData bị Axios gán JSON).

Kiểm thử backend chạy trên PostgreSQL thật (database tạm `legal_assistant_test`), không phải SQLite, vì FTS phụ thuộc `tsvector`, `unaccent`, `pg_trgm`.

1.5 Đối tượng và phạm vi

1.5.1 Đối tượng

- Người dùng: chủ DNNVV, kế toán, nhân sự — hỏi và tra cứu tiếng Việt thường ngày.
- Đối tượng kỹ thuật: pipeline hybrid RAG trên corpus VBQPPL cắt theo Điều; ứng dụng web đa người dùng.

1.5.2 Phạm vi chức năng

Bốn nghiệp vụ: hỏi đáp có trích dẫn, tra cứu cây văn bản, soát xét hợp đồng, lịch tuân thủ. Không OCR file scan, không nộp thuế hộ, không ký sổ, không đại diện tố tụng. Lịch tuân thủ là lịch nhắc, không phải phần mềm kế toán.

1.5.3 Phạm vi dữ liệu và kỹ thuật

Corpus 38 văn bản thuộc thuế, lao động, BHXH, kế toán, hợp đồng, SHTT mức cơ bản, hỗ trợ DNNVV. Không có luật hình sự, không văn bản địa phương. Mô hình và embedding gọi qua

mạng (Gemini); không tự host GPU. Soát xét hợp đồng dùng BackgroundTasks của FastAPI — đủ quy mô đề án, chưa phải hàng đợi phân tán.

1.6 So sánh nhanh với công cụ sẵn có

Bảng 1.1: Ba kiểu công cụ em đối chiếu trước khi chốt hướng.

Tiêu chí	Cổng VBQPPL	Chatbot nguồn	không	Hướng đề án
Cách hỏi	Số hiệu, từ khóa	Câu nói thường		Câu thường hoặc từ cây văn bản
Đối chiếu	Chính là file luật	Không có chỗ bấm		Mỗi căn cứ mở đúng Điều
Bịa số Điều	Không xảy ra	Dễ gặp		Gỡ căn cứ ngoài tập truy hồi
Việc kế toán/NS	Không	Không		Soát hợp đồng, lịch nộp
Độ rộng kho	Rất rộng	Không rõ nguồn		38 văn bản, cố ý hẹp

Cổng nhà nước thắng độ tin cậy và độ phủ. Chatbot thắng cách hỏi. Đề án không cạnh tranh độ phủ; chỗ Em nhắm là ghép cách hỏi tự nhiên với kho hẹp mở được từng Điều, cộng hai việc cổng văn bản không làm: đọc hợp đồng và nhắc mốc nộp.

1.7 Phần em nhận là đóng góp

Em không nhận đóng góp thuật toán mới. Ở mức đề án ngành, ba thứ em nhận là của mình vì chúng không có sẵn trong pipeline thi đấu:

- Ứng dụng nhiều người, nhiều công ty, đăng nhập và cách ly dữ liệu — không còn là script hỏi đáp một lần.
- Tra cứu tiếng Việt không dấu trên Postgres mà đoạn trích vẫn còn dấu; lịch tuân thủ sinh theo hồ sơ từng công ty và dẫn Điều có trong corpus.
- Hệ thống vẫn sống khi thiếu khóa mô hình: tra cứu và lịch dùng được; chỉ hỏi đáp / soát xét báo chưa sẵn sàng.

1.8 Bố cục báo cáo

Chương 2 trình bày cơ sở lý thuyết và lý do chọn công nghệ. Chương 3 phân tích yêu cầu, thiết kế và hiện thực. Chương 4 mô tả thử nghiệm, kiểm thử và đánh giá trên máy làm đề án. Chương 5 kết luận, hạn chế và hướng phát triển.

Chương 2. CƠ SỞ LÝ THUYẾT

Chương này không chép định nghĩa giáo trình cho đủ trang. Em viết lại những khái niệm đúng chỗ mình dùng trong code, kèm lý do chọn hoặc bỏ.

2.1 Xử lý ngôn ngữ tự nhiên trong bài toán hỏi đáp

Hỏi đáp pháp lý trên văn bản là một nhánh của bài toán hỏi đáp miền mở (open-domain QA): đầu vào là câu hỏi ngôn ngữ tự nhiên, đầu ra là câu trả lời kèm (hoặc dựa trên) đoạn văn hỗ trợ. Khác QA trên Wikipedia tiếng Anh, corpus luật Việt Nam có cấu trúc cứng (Chương, Điều, Khoản), thuật ngữ chuẩn hóa cao, và hệ quả sai lệch lớn nếu dẫn nhầm số Điều.

Hai hướng lớn:

- **Close-book:** mô hình trả lời từ tham số đã huấn luyện. Nhanh, nhưng không chỉ được trang luật nào, và dễ bịa khi luật đổi.
- **Open-book / retrieve-then-read:** tìm đoạn liên quan rồi đọc (hoặc sinh) câu trả lời [4, 3].
Đề án theo hướng này.

2.2 Cấu trúc văn bản quy phạm pháp luật Việt Nam

2.2.1 Luật, nghị định, thông tư không nằm trên một trang

Câu “công ty mười hai lao động khai GTGT theo tháng hay theo quý” không nằm gọn trong một Điều của Luật Quản lý thuế. Luật nói nghĩa vụ kê khai tồn tại. Nghị định nói ngưỡng và kỳ. Thông tư nói tờ khai. Nếu corpus chỉ có luật, máy trả lời đúng nguyên tắc và sai thao tác — kiểu sai khó phát hiện vì nghe “có vẻ đúng luật”. Vì vậy em lấy cả ba cấp vào kho [10, 8, 9].

2.2.2 Điều là đơn vị cắt dữ liệu

Văn bản chia Phần – Chương – Mục – Điều – Khoản – Điểm. Em cắt theo **Điều** vì ba lý do thực dụng:

1. Giới hạn hành nghề dẫn chiếu theo Điều, không theo đoạn 400 ký tự thứ n .
2. Một Điều thường đủ nghĩa để đứng trong prompt; cắt giữa câu làm mất chủ ngữ của khoản.
3. Độ dài trung bình khoảng hơn một nghìn ký tự — nhét được vài Điều vào ngữ cảnh mà chưa đầy.

Cắt cả văn bản thành một vector thì vô dụng. Cắt theo số ký tự cố định thì đúng bài NLP chung nhưng hỏng dẫn chiếu pháp lý.

2.2.3 Viện dẫn chéo

Corpus gốc có trường extra: danh sách chỗ một Điều trở sang Điều khác. Em không xây đồ thị tri thức đủ chuẩn trong đồ án (không đủ thời gian). Em lưu và hiện thành liên kết trên trang đọc. Người đang xem Điều 44 Luật Quản lý thuế thấy đường sang Điều nói thời hạn nộp — lúc demo, người không học luật dùng chỗ “nhảy” này nhiều hơn ô chat.

2.3 Truy hồi thưa: BM25

BM25 chấm tài liệu theo tần số từ của câu hỏi, có phạt tài liệu dài và hạ điểm từ xuất hiện khắp nơi [5]:

$$\text{BM25}(Q, D) = \sum_{t \in Q} \text{IDF}(t) \cdot \frac{f(t, D) \cdot (k_1 + 1)}{f(t, D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{\text{avgl}}\right)} \quad (2.1)$$

$$\text{IDF}(t) = \ln \left(\frac{N - n(t) + 0.5}{n(t) + 0.5} + 1 \right) \quad (2.2)$$

Trong đồ án: $k_1 = 1,2$, $b = 0,65$. Em không tối ưu lưới tham số cho đẹp bảng; hơi giảm b vì Điều luật dao động độ dài.

Chỗ BM25 cứu hệ thống: người dùng gõ “59/2020/QH14” hoặc “Điều 23”. Vector để coi hai số Điều gần nhau là gần nghĩa; BM25 khớp mặt chữ.

Chỗ BM25 chịu thua: “sa thải” không xuất hiện trong Điều nói đơn phương chấm dứt — điểm bằng không.

Tiếng Việt không tách từ bằng khoảng trắng. Em dùng underthesea khi có, fallback regex nếu thư viện không nạp được. Chính vì underthesea mà backend ghim Python 3.10.

2.4 Truy hồi dày: embedding và cosine

Mỗi Điều được nhúng thành vector. Độ gần đo bằng cosine:

$$\text{sim}(q, d) = \frac{\mathbf{v}_q \cdot \mathbf{v}_d}{\|\mathbf{v}_q\| \|\mathbf{v}_d\|} \quad (2.3)$$

Sản phẩm dùng gemini-embedding-001, cắt còn 1536 chiều (Matryoshka: giữ đầu vector) cho nhẹ chỉ mục. Đổi mô hình nhúng là đổi số chiều — bắt buộc xóa Chroma và nhúng lại. Em ghi rõ trong cấu hình để khỏi quên.

2.5 Gộp hạng: Reciprocal Rank Fusion

Điểm BM25 và cosine không cùng thước. Cộng thẳng là tự bịa. RRF bỏ điểm, chỉ giữ thứ tự [6]:

$$\text{RRF}(d) = \sum_i \frac{w_i}{k + \text{rank}_i(d)} \quad (2.4)$$

Em lấy $k = 60$, $w_{\text{dense}} = 3$, $w_{\text{BM25}} = 1$. Thêm về vector vì đa số câu DNNVV là câu hỏi nghĩa; BM25 vẫn kéo truy vấn kiểu “Điều 35”. Tỷ lệ 3:1 chọn sau vài vòng thử tay, không phải tối ưu lưới có bài báo.

2.6 RAG, HyDE và rủi ro bịa nguồn

2.6.1 RAG

Mô hình không được phép “nhớ luật” làm nguồn chính. Máy chủ tìm vài Điều, nhét vào prompt, yêu cầu viết từ đó [3]. Lợi ngay: có chỗ dẫn; đổi luật thì nhập văn bản, không huấn luyện lại; so được danh sách số Điều trong câu trả lời với danh sách vừa tìm.

Ba rủi ro em gặp khi chạy thử:

- Máy viết thêm khoản “thường thì doanh nghiệp nên...” không có trong ngữ cảnh.
- Câu đúng nhưng dẫn nhầm Điều — người dùng thấy có căn cứ nên tin.
- Đưa quá nhiều Điều, mô hình lạc. Vì vậy có `llm_filter`. Reranker cross-encoder tắt vì Gemini không có cổng rerank.

2.6.2 HyDE

Câu hỏi ngắn, văn luật dài và khác giọng. HyDE [7] nhờ mô hình viết một đoạn “như thể đã trả lời”, rồi lấy đoạn đó đi tìm vector. Đoạn giả có thể sai nội dung nhưng thường gần giọng luật hơn câu hỏi gốc. Cấu hình đang bật HyDE, tắt viết lại nhiều biến thể — làm cả hai tốn thời gian và hạn mức.

2.7 Chỉ số đánh giá retrieval: vì sao F2

$$F_\beta = (1 + \beta^2) \cdot \frac{P \cdot R}{\beta^2 P + R}, \quad \beta = 2 \quad (2.5)$$

Lấy thừa một Điều, bước lọc sau còn có cơ hội bỏ. Bỏ sót Điều quyết định thì không bước nào cứu. Cuộc thi ưu tiên độ phủ; đề án giữ cùng hướng cho nhất quán, dù sản phẩm không nộp bài thi và **không lấy F2 cuộc thi làm kết quả**.

Exact Match quá chặt cho câu trả lời sinh tự do. ROUGE hữu ích khi có vàng chuẩn câu chữ; đề án chưa gắn nhãn bộ câu nội bộ đủ lớn nên không báo ROUGE giả.

2.8 Word embedding và đặc thù tiếng Việt (nhìn từ chỗ cài)

Trong RAG, embedding không phải để “hiểu luật” theo nghĩa luật học, mà để xếp các Điều gần nghĩa với câu hỏi (hoặc với đoạn HyDE) vào top- k . Với tiếng Việt, ba điểm em chú ý khi chọn và cấu hình:

1. **Tách từ trước BM25.** Không tách thì “lao động” và “lao” “động” thành nhiều. `underthesea` xử lý được phần lớn; chỗ fail thì fallback regex — chấp nhận giảm chất lượng hơn là crash startup.
2. **Dấu thanh.** Vector thường chịu được thiếu dấu tốt hơn BM25. Ngược lại, FTS Postgres của em chủ động bỏ dấu để khớp cách gõ thực tế.
3. **Số chiều và hạn mức.** 3072 chiều đầy đủ của Gemini embedding nặng chỉ mục và đắt hơn khi gọi API. Cắt 1536 chiều là thỏa hiệp: nhẹ một nửa, đủ dùng trên corpus 3.844 Điều.

2.9 Prompt engineering trong đồ án

Em không fine-tune mô hình. Mọi hành vi điều khiển bằng prompt hệ thống và prompt từng nút:

- Intent: chỉ được trả NEXT hoặc SKIP.
- HyDE: viết 5–6 câu giọng luật, không kết luận mức phạt bịa.
- Filter: chỉ trả lời có hoặc không với từng Điều ứng viên.
- Generate: chỉ kết luận từ căn cứ đang có; thiếu thì nói thiếu; mặc định chủ thể là DNNVV trừ khi người dùng nói khác.

Few-shot nằm trong prompt rewrite (ví dụ sa thải / thuế GTGT hộ kinh doanh) nhưng nhánh rewrite đang tắt trên sản phẩm để giữ latency. Nếu bật lại, phải đo thêm hạn mức.

2.10 Full-text search tiếng Việt trên PostgreSQL

PostgreSQL không có từ điển FTS tiếng Việt sẵn như tiếng Anh. Chiến lược đồ án [11]:

1. Bỏ dấu bằng `unaccent`, cấu hình `simple`.
2. Vì `unaccent` chỉ STABLE, phải bọc `immutable_unaccent` mới tạo được cột `generated tsvector`.
3. Ba tầng: lọc theo số Điều; `ts_rank`; `pg_trgm` ngưỡng 0,3 khi không có hit.
4. Không dùng `ts_headline` để bôi đậm — hàm chạy trên cột mất dấu. Backend trả token; React bôi trên chữ gốc.

2.11 Công nghệ hiện thực và lý do chọn

Bảng 2.1: Công nghệ gắn vào sản phẩm.

Thứ dùng	Nhóm	Việc trong đồ án
React TypeScript	19, Giao diện	SPA sau đăng nhập
Vite + nginx	Đóng gói FE	Dev proxy / production proxy / api
Tailwind CSS 4	Trang trí	Không kéo bộ component nặng
TanStack Query	Cache client	Luật, poll soát xét
Zustand	Phiên	User + khôi phục token
FastAPI	Máy chủ	REST + SSE bất đồng bộ
SQLAlchemy 2 + Alembic	ORM / migration	Schema phiên bản hóa
PostgreSQL 17	CSDL	Nghiệp vụ + FTS
ChromaDB	Vector	Cosine trên Điều
LangGraph	Điều phối	Bảy nút pipeline
slowapi	Rate limit	Bảo vệ hạn mức mô hình
Docker Compose	Cụm	Postgres, backend, frontend

FastAPI. Phần lớn thời gian request là chờ mạng. SSE có sẵn. OpenAPI sinh từ Pydantic — lúc demo đưa /docs.

SPA, không Next.js. Code thi đấu từng là Next.js. Em đổi vì đề bài yêu cầu React và vì app nằm sau đăng nhập, không cần SSR cho SEO. Đọc SSE bằng Fetch (không EventSource: không POST, không gắn Bearer).

Postgres + Chroma + BM25 RAM. Quan hệ và giao dịch — Postgres. Vector — Chroma. BM25 dựng trong RAM, cache theo dấu vân tay tokenizer để startup không tách lại 3.844 Điều mỗi lần.

LangGraph. Bảy bước, có nhánh thoát, mỗi bước bắn tiến độ SSE.

argon2 + JWT ngắn hạn. Không có Redis cho session server-side. Access 15 phút, refresh 7 ngày, phân biệt bằng field type.

SSE thay WebSocket. Hỏi đáp một chiều. SSE đi trên HTTP, nginx proxy dễ.

2.12 Bảng tổng kết: vai trò – thách thức – hướng xử lý

Cách trình bày dưới đây học theo cấu trúc khóa luận mẫu (vai trò / thách thức / giải pháp), nhưng nội dung là của đồ án pháp lý DNNVV.

2.13 Việc người khác đã làm, em lấy gì

Em lấy ý RAG và dense retrieval [3, 4]; BM25 và RRF là công cụ [5, 6]; HyDE để kéo giọng câu hỏi gần văn luật [7]. Bài chatbot tuyển sinh [2] là mốc: hướng RAG chạy được trên dữ liệu hẹp tiếng Việt.

Bảng 2.2: Tổng kết khối truy hồi hybrid.

Hạng mục	Nội dung
Vai trò	Đưa đúng tập Điều liên quan vào prompt trước khi sinh câu.
Thách thức	Lệch từ vựng người–luật; số hiệu dễ bị vector “làm tròn”; hạn mức nhúng.
Hướng xử lý	BM25 + Chroma + RRF 3:1; HyDE; batch embedding 64 kèm backoff; cache BM25.

Bảng 2.3: Tổng kết khối sinh câu và hậu kiểm trích dẫn.

Hạng mục	Nội dung
Vai trò	Viết câu tiếng Việt có thể đối chiếu được với Điều gốc.
Thách thức	Mô hình bịa số Điều; diễn giải sai trên Điều đúng; đột hạn mức.
Hướng xử lý	Prompt siết căn cứ; <code>llm_filter</code> ; gỡ citation ngoài tập truy hồi; nhiệt độ 0; thiếu key thì 503 chữ không sập cụm.

Bảng 2.4: Tổng kết khối nghiệp vụ DNNVV (hợp đồng + lịch).

Hạng mục	Nội dung
Vai trò	Biến RAG thành việc kế toán/nhân sự làm hàng tuần, không chỉ chat.
Thách thức	PDF ảnh; hạn chót luật đổi; cách ly dữ liệu nhiều công ty.
Hướng xử lý	Cắt điều khoản + reuse retrieval; 12 rule có <code>legal_refs</code> ; cách ly theo organization/user; từ chối PDF không có lớp chữ lúc tải.

Chỗ đồ án khác: kho có cấu trúc Điều–Chương, FTS không dấu vẫn hiện có dấu, lịch tuân thủ gắn Điều thật, tách sản phẩm Gemini khỏi bài thi mô hình mở. Phần hiện thực nằm ở Chương 3; phần đo trên máy em nằm ở Chương 4.

Chương 3. PHÂN TÍCH, THIẾT KẾ VÀ HIỆN THỰC

3.1 Hệ thống nhìn từ phía người bấm

Người dùng tạo tài khoản, khai tên công ty, mã số thuế, số lao động, kỳ kê khai GTGT. Bốn việc chính sau đó:

Hỏi. Gõ câu hỏi, thấy từng bước (đang tìm, đang lọc, đang viết), thấy từng chữ hiện ra, thấy danh sách Điều bấm được. Hội thoại cũ nằm cột trái, sống sót khi restart máy chủ.

Đọc luật. Lọc theo lĩnh vực, vào một văn bản, đi từ Chương xuống Điều, bấm viện dẫn chéo, hoặc bấm “Hỏi AI về Điều này” để câu hỏi được điền sẵn kèm số hiệu.

Đưa hợp đồng. Kéo file PDF/DOCX/TXT, tối đa 10 MB. Máy trích chữ ngay (lỗi file lộ lúc tải, không đợi). Bấm soát xét thì chạy nền; trang hỏi trạng thái mỗi vài giây. Kết quả là bảng: điều khoản, mức cao/trung bình/thấp, đề xuất sửa, căn cứ.

Nhìn lịch. Các mốc trong năm, nhóm theo tháng, đánh dấu xong hoặc bỏ qua. Sinh lại khi đổi số lao động hay đổi kỳ GTGT.

Người quản trị (tài khoản đầu tiên khi bảng user còn trống được gán vai trò này) vào được kho văn bản, danh sách user, và trang lab chạy bộ câu hỏi theo khuôn `results.json`.

3.2 Yêu cầu

3.2.1 Chức năng

Bảng 3.1: Yêu cầu chức năng em chốt trước khi viết API.

Mã	Việc	Ghi chú khi cài
YC-01	Đăng ký, đăng nhập, làm mới token, đổi mật khẩu	Access 15 phút, refresh 7 ngày, hai loại token không dùng lẫn
YC-02	Hỏi đáp có trích dẫn và stream	Ghi câu hỏi vào CSDL trước khi gọi pipeline
YC-03	Lịch sử hội thoại	Đổi tên, lưu trữ, xóa; không nhìn hội thoại người khác (trả 404)
YC-04	Danh mục và cây văn bản	Số hiệu có dấu / nên định tuyến phải kiểu path
YC-05	Tìm toàn văn không dấu	Ba tầng: số Điều, <code>ts_rank</code> , trigram
YC-06	Tải và soát xét hợp đồng	Trích chữ lúc tải; chấm điểm lúc soát xét
YC-07	Lịch tuân thủ	Idempotent: gọi sinh lại không xóa việc đã đánh dấu xong
YC-08	Quản trị corpus và user	Nhập JSON, reindex, khóa tài khoản
YC-09	Lab thi đấu	Chỉ admin, giữ khuôn nộp bài cũ

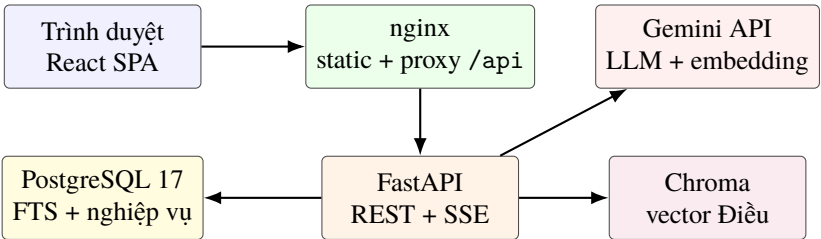
3.2.2 Phi chức năng

Thời gian: trang tra cứu phải trả trong khoảng dưới một giây trên máy em; hỏi đáp chấp nhận hàng chục giây nếu hiện tiến độ. Bảo mật: mật khẩu argon2, CORS không phải *, giới hạn số lần gọi chat. Sẵn sàng: thiếu embedding thì phần còn lại vẫn sống. Kiểm thử: không chấp nhận test chỉ chạy trên SQLite.

Bảng 3.2: Ngưỡng phi chức năng em tự đặt khi viết code.

Nhóm	Ngưỡng	Cách kiểm
Độ trễ tra cứu	< 1 giây trên máy em với FTS	Gọi /laws/search
Độ trễ hỏi đáp	Chấp nhận hàng chục giây nếu có SSE tiến độ	Quan sát UI
Bảo mật mật khẩu	argon2id, tối thiểu 8 ký tự	Test đăng ký
Cách ly dữ liệu	Hội thoại theo user; hợp đồng/lich theo org	Test 404
Sẵn sàng suy giảm	Thiếu embedding: health 200, chat 503	Smoke Docker
Giới hạn tần suất	Chat 20/phút; upload 30/giờ	slowapi
Kiểm thử	Không chấp nhận test chỉ xanh trên SQLite	CI/pytest local

3.3 Kiến trúc ba lớp



Hình 3.1: Ba lớp triển khai: trình duyệt → nginx → FastAPI; PostgreSQL, Chroma và Gemini nằm phía sau API.

Lúc cài bằng Docker, trình duyệt chỉ nói chuyện với cổng frontend. Nginx vừa đưa file tĩnh vừa chuyển /api sang backend, nên không có chuyện CORS trên môi trường đó. Lúc em lập trình, Vite làm đúng việc chuyển tiếp ấy trên cổng 5173.

Logic RAG không nằm trong file router. Router chỉ nhận request, ghi hội thoại, gọi agent. Cùng một hàm truy hồi được soát xét hợp đồng gọi lại. Ban đầu em định copy prompt sang module hợp đồng cho “nhanh”; rồi nhận ra sửa một chỗ sót một chỗ, nên tách sớm.

BM25 không để Postgres. Khởi động backend: đọc Điều từ Postgres, nếu file cache khớp dấu vân tay tokenizer thì nạp cache, không thì tách từ lại và ghi cache. Vector: nếu manifest (model, số chiều, số record) lệch thì nhúng lại, có chia lô và nghỉ dần khi bị 429.

3.4 Use case

Bảng 3.3: Tác nhân.

Tác nhân	Người nào	Được làm
Chủ doanh nghiệp	Người mở tài khoản công ty	Cả bốn nghiệp vụ trong công ty mình
Kế toán, Nhân sự	Người được mời vào cùng công ty	Cùng bốn nghiệp vụ; không vào trang quản trị
Quản trị	Tài khoản đầu, hoặc được đổi role	Thêm quyền corpus, user, lab
Khách	Chưa đăng nhập	Chỉ đăng ký / đăng nhập

Bảng 3.4: Danh sách use case.

Mã	Tên	Ai
UC-01	Tạo tài khoản kèm hồ sơ công ty	Khách
UC-02	Đăng nhập, làm mới phiên	Mọi vai trò đã có tài khoản
UC-03	Hỏi đáp pháp luật	Chủ, kế toán, nhân sự
UC-04	Xem và đổi tên hội thoại	Người sở hữu hội thoại
UC-05	Đọc văn bản theo cây Chương–Điều	Chủ, kế toán, nhân sự
UC-06	Tìm chữ trong corpus	Chủ, kế toán, nhân sự
UC-07	Tải hợp đồng và chạy soát xét	Người thuộc một công ty
UC-08	Đọc kết quả soát xét	Người cùng công ty
UC-09	Xem lịch, đánh dấu xong	Người thuộc một công ty
UC-10	Nhập thêm văn bản, dựng lại chỉ mục	Quản trị
UC-11	Khóa / đổi vai trò user	Quản trị
UC-12	Chạy bộ câu hỏi lab	Quản trị

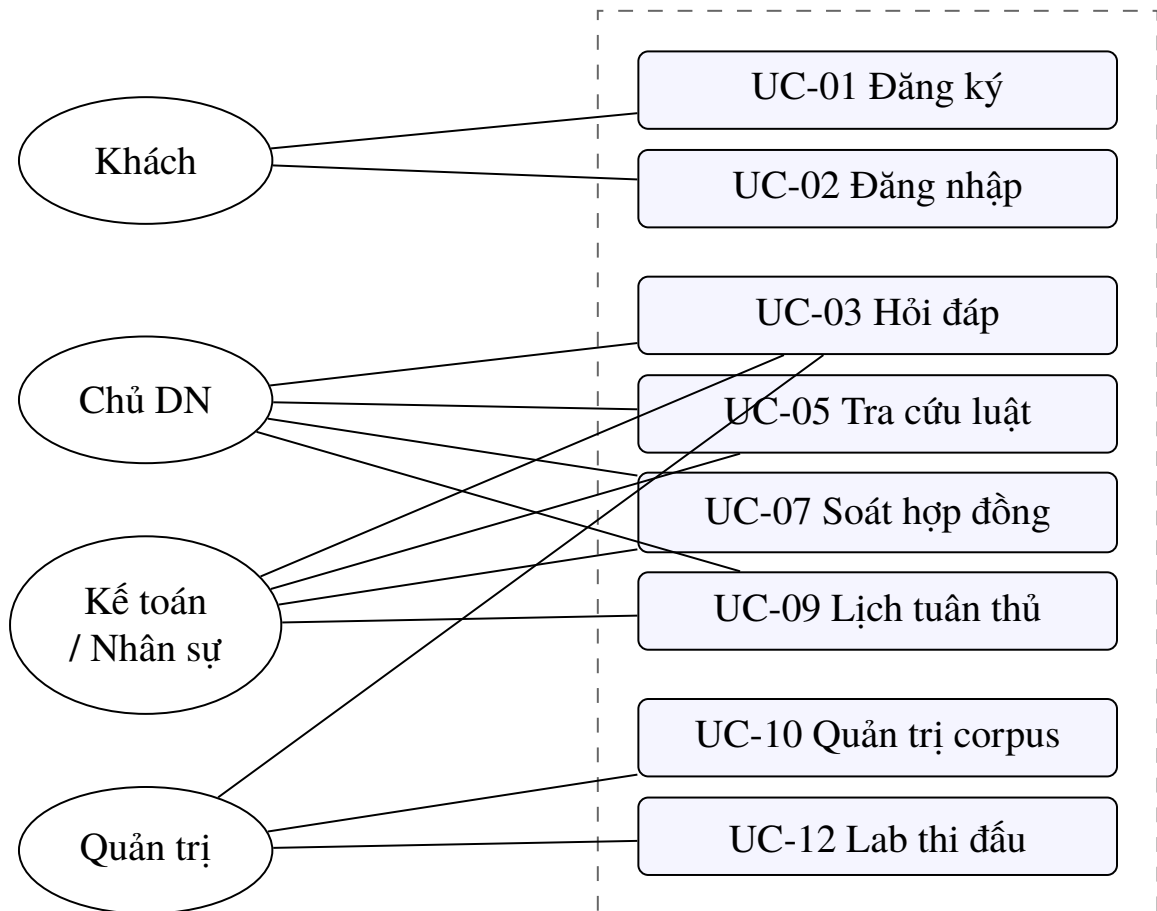
3.4.1 Đặc tả bổ sung UC-05 và UC-09

3.5 Bảy bước hỏi đáp

Đồ thị LangGraph có đúng bảy nút, tên trong code giữ tiếng Anh vì em không muốn đổi identifier khi port từ bài thi. Thứ tự chạy cố định, trừ hai nhánh thoát.

1. `analyze_intent`. Mô hình chỉ được trả NEXT hoặc SKIP. Câu chào, hỏi giờ, hỏi “ăn gì” dừng tại đây, không tốn nhúng. Lab thi đầu bỏ qua nút này để mọi câu trong bộ đề đều đi hết đường.
2. `prepare_retrieval_query`. Cấu hình sản phẩm: HyDE bật, viết lại truy vấn nhiều biến thể tắt. Em thử bật cả hai một lần; vừa chậm vừa đốt hạn mức, trong khi lúc hỏi tay HyDE

Hệ thống trợ lý pháp lý



Hình 3.2: Sơ đồ use case. Chủ / kế toán / nhân sự dùng bốn nghiệp vụ; quản trị thêm corpus và lab.

Bảng 3.5: Đặc tả UC-03 — Hỏi đáp.

Trường	Nội dung
Mã	UC-03
Tóm tắt	Người dùng gửi một câu tiếng Việt, nhận câu trả lời kèm Điều dẫn.
Trước khi làm	Đã đăng nhập. Chỉ mục vector sẵn thì hỏi được; chưa sẵn thì nhận 503 và được hướng sang trang tra cứu.
Sau khi làm	Câu hỏi và câu trả lời nằm trong hội thoại, kèm JSON trích dẫn và vết các bước.
Luồng chính	1. Gửi câu. 2. Máy tạo hội thoại nếu chưa có, lấy vài lượt cũ. 3. Ghi câu hỏi. 4. Chạy bảy bước, mỗi bước một sự kiện SSE. 5. Ghi câu trả lời khi có sự kiện kết quả. 6. Trình duyệt đổi URL sang /chat/{id}.
Nhánh	Câu không phải pháp luật: thoát ở bước ý định, không tồn truy hồi. Không tìm ra Điều: nói không có căn cứ, không bịa.
Lỗi	Mất kết nối: câu hỏi đã lưu, gửi lại được. Hết hạn mức mô hình: báo lỗi rõ, không xóa hội thoại. Quá số lần gọi/phút: 429.

Bảng 3.6: Đặc tả UC-07 — Soát xét hợp đồng.

Trường	Nội dung
Mã	UC-07
Tóm tắt	Tải file, chạy soát xét nền, đọc bảng rủi ro.
Trước khi làm	Đã gắn công ty. File .pdf, .docx hoặc .txt, ≤ 10 MB.
Sau khi làm	Tài liệu done hoặc failed. Có thể chạy lại.
Luồng chính	1. Tải, trích chữ, trạng thái ready. 2. Bấm soát xét, trả 202 ngay. 3. Nền: cắt điều khoản, mỗi khoản đi truy hồi, chấm rủi ro. 4. Trang hỏi lại mỗi 2,5 giây.
Nhánh	PDF ảnh (không có lớp chữ): fail lúc tải, không lúc soát xét — em cố tình vậy để người dùng biết ngay. Quá nhiều điều khoản: cắt theo ngưỡng cấu hình, ghi trong tóm tắt.
Lỗi	Sai định dạng: 400. Mất mô hình giữa chừng: failed, bấm thử lại.

Bảng 3.7: Đặc tả UC-05 — Tra cứu và đọc văn bản.

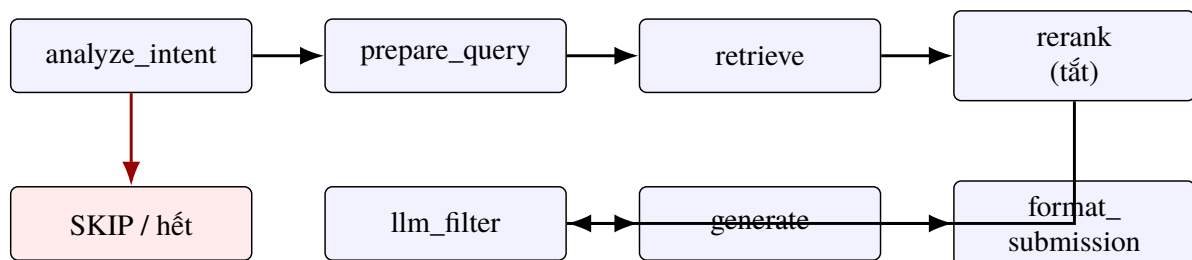
Trường	Nội dung
Mã	UC-05
Tóm tắt	Duyệt cây Chương–Điều, tìm chữ (kể cả không dấu), mở viện dẫn chéo.
Trước khi làm	Đã đăng nhập. Corpus đã nạp Postgres (không cần Chroma).
Sau khi làm	Người dùng đọc được nguyên văn Điều; có thể chuyển sang hỏi đáp.
Luồng chính	1. Mở /laws. 2. Lọc lĩnh vực hoặc chọn số hiệu. 3. Xem cây. 4. Mở Điều. 5. (Tuỳ chọn) bấm viện dẫn hoặc “Hỏi AI về Điều này”.
Nhánh tìm	Gõ ô tìm: ưu tiên số Điều; rồi FTS; rồi trigram nếu không hit.
Lỗi	Số hiệu sai đường dẫn: 404. Query rỗng: không gọi API.

Bảng 3.8: Đặc tả UC-09 — Lịch tuân thủ.

Trường	Nội dung
Mã	UC-09
Tóm tắt	Xem, đánh dấu, sinh lại các mốc nghĩa vụ theo hồ sơ công ty.
Trước khi làm	User thuộc một organization có vat_period, số lao động.
Sau khi làm	Task phản ánh đúng rule áp dụng; việc đã tick không bị xóa khi sinh lại.
Luồng chính	1. Đăng ký/cập nhật hồ sơ. 2. Hệ thống sinh cửa sổ 3 tháng trước + 12 tháng tới. 3. User xem theo tháng, đánh dấu xong/bỏ qua. 4. Đổi hồ sơ thì bấm sinh lại.
Nhánh	Công ty < 10 lao động: không có rule ATVSLĐ. GTGT tháng/quý loại trừ lẫn nhau.
Lỗi	Chưa gắn công ty: API từ chối. Rule thiếu legal_refs: không seed (em bắt trong code).

đã kéo được Điều đúng giọng luật. Nếu gọi HyDE lỗi (hết hạn mức, timeout), nút này nuốt lỗi và tìm bằng câu gốc — không được phép làm hỏng cả lượt hỏi chỉ vì một đoạn giả.

3. `retrieve`. BM25 và Chroma chạy song song, gộp RRF, $k_{\text{RRF}} = 60$, trọng số dense/BM25 = 3/1, lấy top $k = 8$ Điều.
4. `rerank`. Trong file cấu hình đang tắt. Gemini không có cổng rerank; chỗ gắn Qwen reranker cục bộ vẫn để đó nếu ai đó bật lab.
5. `llm_filter`. Tối đa 3 Điều lọc song song, giữ tối thiểu 2 (tránh lọc sạch rồi không còn gì để viết). Mỗi ứng viên được hỏi “có dính câu hỏi này không”, chỉ được trả có hoặc không.
6. `generate_answer`. Nhiệt độ 0, tối đa 4096 token, stream từng token xuống SSE. Prompt hệ thống siết: chỉ kết luận từ căn cứ đang có; thiếu căn cứ thì nói thiếu, không bịa mức phạt.
7. `format_submission`. Regex lấy các cụm “Điều n ” trong câu trả lời, đối chiếu với danh sách id vừa truy hồi. Số không có trong danh sách bị gỡ khỏi mảng citation gửi xuống giao diện. Câu chữ giữ nguyên — em không xóa luôn đoạn mô hình đã viết, vì xóa lung tung dễ làm câu sượng; chỉ không cho bấm vào một nguồn giả.



Hình 3.3: Bảy nút LangGraph: nhánh SKIP thoát sớm; rerank tắt trên sản phẩm Gemini.

3.6 Corpus

Kho nằm trong `data/base_data.json`: 3.844 bản ghi. Tập kê khai `corpus/law_manifest.json` gắn lĩnh vực cho từng số hiệu, dùng để đổ bảng laws mỗi lần backend lên.

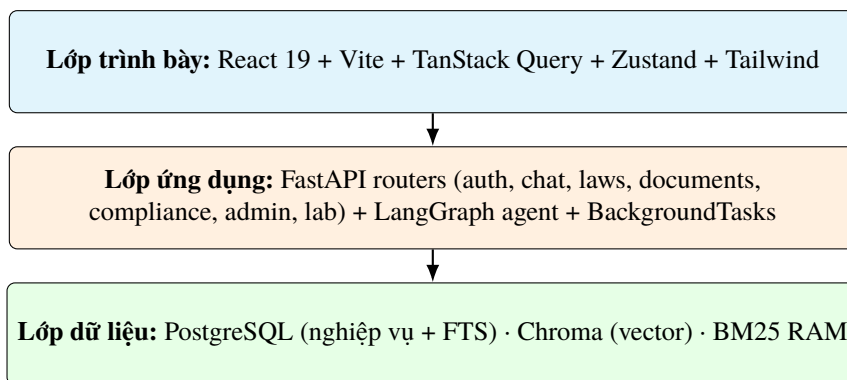
Thuế nhiều nhất vì lịch tuân thủ và câu hỏi hàng ngày của kế toán tập trung ở đó. Có cả luật cũ và luật mới cùng tên (ví dụ Luật GTGT 13/2008 và 48/2024): em giữ cả hai số hiệu vì hợp đồng và tình huống thực tế vẫn dẫn bản đang có hiệu lực tại thời điểm ký. Việc “chọn đúng bản còn hiệu lực” em chưa tự động hóa — đây là hạn chế, không giấu.

Cắt theo Điều đã nói ở Chương 2. Trường extra giữ viện dẫn. Bảng `legal_knowledge_records` giữ khóa id nguyên từ file JSON, không tự tăng, để Chroma và Postgres chỉ cùng một mã.

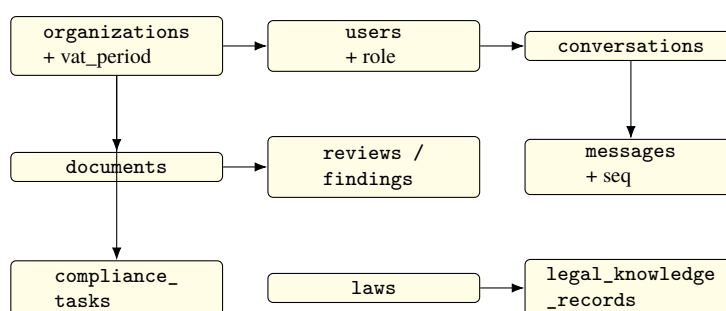
Bảng 3.9: Đếm nhanh theo loại và theo lĩnh vực (một văn bản vào đúng một lĩnh vực).

Loại	Số	Lĩnh vực	Số
Luật	19	Thuế	12
Nghị định	14	Lao động	4
Thông tư	3	Sở hữu trí tuệ	4
Bộ luật	2	Doanh nghiệp	3
		Dân sự – hợp đồng	3
		Kế toán – tài chính	3
		Còn lại (BHXH, ATVSLĐ, NTD, đất đai, hỗ trợ DNNVV)	9
Cộng	38	Cộng	38

3.7 Mô hình dữ liệu



Hình 3.4: Kiến trúc ba lớp: trình bày, ứng dụng, dữ liệu.



Hình 3.5: Lược đồ quan hệ chính. Hội thoại thuộc user; hợp đồng và lịch thuộc organization.

3.7.1 Ba chỗ em làm sai rồi sửa

Thứ tự tin nhắn. Câu hỏi và câu trả lời ghi trong một transaction HTTP. `now()` của Postgres là mốc đầu transaction, hai dòng cùng `created_at`. Lúc tải lại hội thoại, thứ tự bị đảo ngẫu nhiên. Sửa: cột `seq` tăng đơn điệu, API sort theo `seq`.

Bảng 3.10: Bảng. Tên cột chỉ liệt kê những cột em từng vấp khi cài.

Bảng	Ghi chú hiện thực
organizations	vat_period, employee_count quyết định rule nào được sinh. tax_code unique.
users	Email lower-case. Role text, không dùng ENUM native Postgres để khỏi ALTER TYPE mỗi lần thêm giá trị.
conversations	Tiêu đề cắt từ câu hỏi đầu, tối đa 80 ký tự, cắt tại khoảng trắng.
messages	Cột seq identity. Không sort theo created_at.
documents	File trên đĩa tên UUID, không lấy tên gốc.
contract_reviews, contract_findings	Một tài liệu nhiều lượt soát xét, không ghi đè.
compliance_rules / tasks	Unique (công ty, rule, kỳ) để sinh lại không nhân bản.
legal_knowledge_ records	Generated: article_number, doc_ref, search_vector.
laws	Đổ từ manifest + đếm Điều, mỗi lần startup.
audit_logs	Đăng ký, đăng xuất, đổi mật khẩu, tải file, reindex.

404 thay vì 403. Nếu trả 403 khi xem hội thoại người khác, kẻ xấu biết id đó tồn tại. Em trả 404 giống “không có”. Test có case hai tài khoản.

InMemorySaver. LangGraph giữ lịch sử trong RAM. Restart mất. Hai worker mỗi bên một nửa ký ức. Đổi: đọc N lượt từ Postgres nhét vào state. `short_memory.max_turns = 6`.

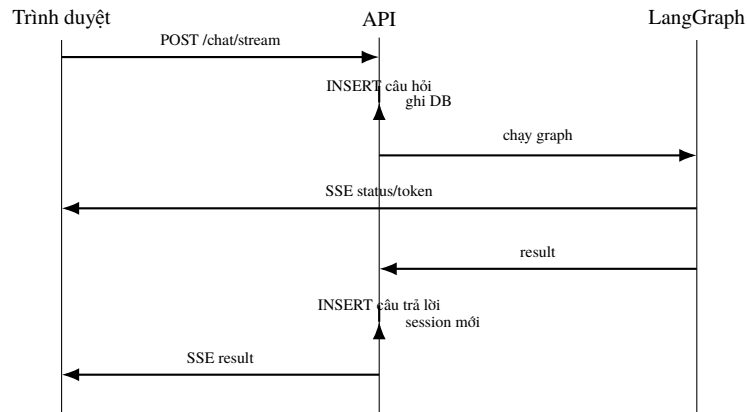
3.8 Tìm chữ trên tiếng Việt

Hàm `unaccent` mặc định là STABLE, không dùng trong generated column. Em bọc `immutable_unaccent` trong `init.sql`, chạy bằng superuser lúc tạo cụm, Alembic không đụng tới (user ứng dụng không đủ quyền CREATE EXTENSION).

Ba tầng tìm, theo thứ tự:

1. Câu có dạng “Điều 12” thì lọc thẳng `article_number`.
2. `tsquery` kiểu prefix (doanh:*) trên `search_vector`, xếp `ts_rank_cd`.
3. Nếu không ra dòng nào, `pg_trgm` với ngưỡng 0,3 trên tiêu đề Điều và tên văn bản — cứu gở sai.

Hàm `ts_headline` chạy trên cột đã bỏ dấu, trả về đoạn không dấu. Em không dùng. Backend gửi list token; React bôi <mark> trên chữ gốc bằng so khớp bỏ dấu phía client. Việc này em viết test riêng vì dễ lệch dấu thanh.



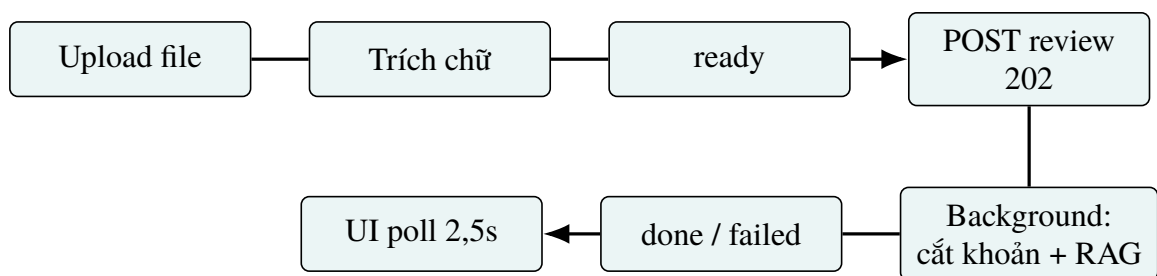
Hình 3.6: Luồng hỏi đáp: ghi câu hỏi trước, stream SSE, ghi câu trả lời ở session CSDL riêng.

3.9 Luồng thời gian thực và lưu hội thoại

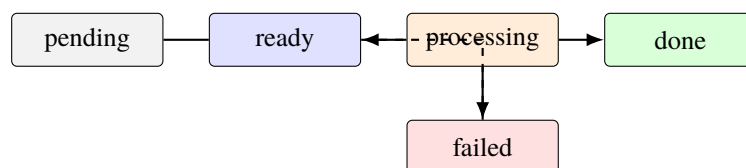
Ba loại sự kiện: status, token, result. Có heartbeat khi một bước kéo dài, để proxy khỏi cắt kết nối. Client không dùng EventSource.

3.10 Hợp đồng

Cắt điều khoản bằng regex tiêu đề “Điều n”, “Article n”, hoặc dòng số kiểu “5. Thanh toán”. Ưu tiên đuôi file vì trình duyệt hay khai application/octet-stream. Khoản dưới 80 ký tự ghép vào khoản trước — thường là tiêu đề mục. Khoản dài quá 6000 ký tự bị cắt để khỏi vượt ngưỡng cảnh. Tối đa 60 khoản một lượt; hợp đồng mẫu DNNVV em thử (vài trang) nằm dưới ngưỡng này. Mỗi khoản gọi lại cùng hàm truy hồi của hỏi đáp, lấy 4 Điều, rồi mô hình gán mức cao / trung bình / thấp / thông tin. Điểm tổng 0–100 từ trọng số mức. PDF không có lớp chữ (scan) fail lúc tải, có câu tiếng Việt bảo cần OCR — không đợi đến lúc bấm soát xét mới báo.



Hình 3.7: Luồng soát xét hợp đồng: trả 202 ngay, xử lý nền, UI poll trạng thái.



Hình 3.8: Máy trạng thái tài liệu: pending → ready → processing → done/failed.

Lúc viết frontend, Axios gán sẵn Content-Type: application/json, gửi FormData là

hông biên multipart. Em gỡ header đó khi body là FormData. Đây là lỗi chỉ lộ khi bấm tải file, test API bằng `httpx files=` thì không dính.

3.11 Lịch tuân thủ

12 rule nằm trong code Python, seed mỗi lần backend lên. Em cố tình không nhét hạn chót vào file cấu hình YAML: rule gắn `legal_refs` (số hiệu + tên văn bản + số Điều). Nếu một hạn chót đổi vì luật mới, người sửa phải đổi cả refs, không được chỉ sửa ngày.

Bảng 3.11: Mười hai rule em cài, và điều kiện hồ sơ để rule được sinh.

Mã	Kỳ	Điều kiện / hạn chót em lấy
VAT_MONTHLY	Tháng	Chỉ khi khai GTGT tháng; ngày 20 tháng sau
VAT_QUARTERLY	Quý	Chỉ khi khai quý; cuối tháng đầu quý sau
CIT_PROVISIONAL	Quý	Mọi công ty; ngày 30 tháng đầu quý sau
CIT_FINALIZATION	Năm	Cuối tháng 3
PIT_FINALIZATION	Năm	Có từ 1 lao động; cuối tháng 3
BUSINESS_LICENSE_FEE	Năm	30/01
SOCIAL_INSURANCE_MONTHLY	Tháng	Có lao động; cuối tháng
LABOUR_REPORT_H1 / H2	Năm	05/06 và 05/12
LABOUR_SAFETY_REPORT	Năm	Từ 10 lao động; 10/01 năm sau
FINANCIAL_STATEMENT	Năm	Cuối tháng 3
INVOICE_USAGE_CHECK	Quý	Rà hóa đơn điện tử, ngày 15 tháng sau quý

Sinh cửa sổ 3 tháng trước (để thấy việc đã quá hạn) và 12 tháng tới. Khóa duy nhất (công ty, rule, kỳ) nên bấm “Sinh lại lịch” sau khi đổi số lao động không xóa việc đã tick xong. Công ty 12 người, khai quý: lúc đăng ký em đếm được 39 task — có GTGT quý, không có GTGT tháng, có báo cáo an toàn lao động vì đủ ngưỡng 10 người. Công ty 3 người khai tháng thì đảo lại: có rule tháng, không có báo cáo an toàn. Test backend bắt đúng hai nhánh này.

Hạn chế thẳng thắn: hạn chót em lấy từ Điều trong corpus tại thời điểm thu thập. Luật sửa sau đó mà chưa nhập văn bản mới thì lịch vẫn nhắc ngày cũ. Hệ thống không tự đọc công báo.

3.12 Đăng nhập, phiên làm việc, bảo mật

Mật khẩu băm `argon2id` (passlib). JWT thuật toán `HS256`, hai loại token, payload có `sub`, `type`, `iat`, `exp`, `jti`. Access sống 15 phút, refresh 7 ngày. Gọi API bằng refresh token bị từ chối vì `type` không khớp — em từng thử nhầm hai token lúc viết `interceptor`, đúng ra lỗi đó.

Tài khoản đăng ký đầu tiên, khi bảng `users` còn trống, được gắn `admin`. Cờ `bootstrap_first_admin` bật mặc định. Không có tài khoản “`admin/admin`” cứng trong image, vì em không

muốn mật khẩu mẫu nằm trong Docker Hub sau này.

Phía trình duyệt, Axios gắn sẵn Authorization. Gặp 401, một lần refresh được chia cho mọi request đang chờ (một Promise chung), tránh gọi /auth/refresh song song rồi làm hỏng phiên. Endpoint auth không đi vào vòng refresh, kéo login sai mật khẩu cũng bị “làm mới token”. Token nằm localStorage: lựa chọn đơn giản cho đề án, em biết rủi ro XSS; CSP và cookie HttpOnly em chưa làm.

CORS: danh sách origin, allow_credentials=true nên không được *. Trên Docker, nginx và API cùng origin nên CORS gần như không chạy; danh sách chủ yếu phục vụ Vite lúc lập trình (localhost:5173).

Giới hạn tần suất: chat 20 lần/phút, tải file 30 lần/giờ, theo từng user (hoặc IP nếu chưa đăng nhập). User thường gọi /api/v1/admin/* nhận 403. Hội thoại người khác: 404, đã nói ở trên.

Bốn vai trò owner, accountant, hr, admin. Hợp đồng và lịch tuân thủ cách ly theo organization_id. Hội thoại cách ly theo user_id — kế toán và chủ cùng công ty không đọc được chat của nhau. Em chọn vậy vì câu hỏi pháp lý đôi khi dính lương, kỷ luật, chuyện không muốn cả phòng biết.

3.13 Giao diện

Frontend là một SPA. Người chưa đăng nhập chỉ vào /login và /register. Người đã đăng nhập bị đẩy khỏi hai trang đó. Route cần phiên chờ restore() đọc token xong mới quyết định, nếu không sẽ nháy sang login rồi nháy lại — em gặp đúng lỗi đó tuần đầu.

Số hiệu văn bản kiểu 45/2019/QH14 chứa dấu gạch chéo. React Router mặc định cắt path theo /, nên em bắt phần còn lại bằng /laws/:lawId/*. Backend cũng nhận path, không nhận query một đoạn, vì cùng lý do.

Trang quản trị và lab lazy() tách bundle: user thường không tải code chạy bộ câu hỏi. TanStack Query tắt refetchOnWindowFocus — corpus không đổi mỗi lần đổi tab, refetch chỉ tổ làm nhấp nháy cây Điều.

Ảnh chụp bố cục từng trang (hỏi đáp, tra cứu, hợp đồng, lịch) đặt ở Chương 4 khi đánh giá giao diện, tránh trùng hình trong báo cáo.

Trang hồ sơ cho sửa số lao động và kỳ GTGT; lưu xong lịch được sinh lại. Lab: kéo file JSON bộ câu, xem tiến độ, tải results.json đúng khuôn cũ.

Em không kéo bộ component thịnh hành. Nút, ô, thẻ viết tay cho gọn bundle. Nền tối, chữ đủ tương phản để đọc Điều dài. Giao diện chưa tối cho điện thoại; thanh bên hội thoại trên màn hẹp sẽ chật. Em ghi vào hạn chế, không nhận đây là giao diện đã chạy tốt trên điện thoại.

3.14 Nhúng vector, cache BM25, và chuyển file .env

Mỗi lần đổi model nhúng hoặc đổi số chiều, backend so manifest trong thư mục Chroma với cấu hình hiện tại. Lệnh thì nhúng lại. Lô 64 Điều, nghỉ dần khi gặp 429 (đợi 2 giây, nhân đôi, trần 60

giây, tối đa 6 lần). Gói miễn phí Google dễ đụng trần nếu nhúng 3.844 Điều một mạch; không có backoff thì lần đầu cài gần như chắc fail giữa chừng.

BM25 tách từ bằng underthesea (fallback regex). Kết quả ghi file cache theo dấu vân tay tokenizer. 3.844 Điều tách lại mỗi lần restart thì startup kéo dài hàng phút; cache là chỗ em vá sau khi đo, không phải ý tưởng có từ ngày đầu.

Một chỗ em mất buổi chiều: Docker Compose chỉ đọc .env ở thư mục gốc dự án. File backend/.env mà em điền khóa lúc lập trình **không** được compose nạp. Backend trong container cứ báo thiếu key dù máy host đã có key. Sửa: copy khóa lên file gốc, ghi rõ trong README. Phụ lục A lặp lại đúng chỗ này vì em biết người chấm (và cả em vài tuần sau) sẽ dẫm lại.

Thiếu khóa: healthcheck vẫn xanh, /health trả ổn, đăng nhập và tra cứu chạy. Hối đáp trả 503 tiếng Việt, gợi ý dùng trang luật. Đây là chỗ thầy góp ý sớm; bản phác của em từng raise lúc import embedding client, kéo theo cả cụm không lên.

3.15 Đóng gói ba dịch vụ

Ba container trên một mạng cầu. Postgres 17, healthcheck pg_isready, volume riêng, init.sql tạo extension unaccent, pg_trgm và hàm immutable_unaccent. Backend đợi Postgres healthy rồi mới lên, healthcheck HTTP /health, start_period 60 giây vì lần đầu Alembic và nạp catalog luật chậm. Frontend là nginx phục vụ bản Vite đã build, proxy /api sang backend, VITE_API_BASE_URL để trống để SPA gọi đường dẫn tương đối — cùng origin, hết chuyện CORS trên môi trường cài đặt.

Cổng ra máy host: web 5173, API 8023, Postgres 23432 (không 5432, tránh đụng Postgres sẵn có trên máy em). Corpus và data/ mount read-only. chroma_db và uploads là volume, sống sót khi rebuild image.

3.16 Tóm tắt hiện thực theo module

Bảng 3.12: Ánh xạ module code ↔ nghiệp vụ.

Thư mục / router	Việc
routers/auth	Đăng ký, login, refresh, đổi mật khẩu
routers/legal	+ SSE hỏi đáp, bấm nút LangGraph
agents/ routers/laws	+ Catalog, cây, FTS ba tầng
legal/search	
routers/documents	+ Upload, trích chữ, soát xét nền
contracts/ routers/compliance	12 rule, sinh task
+ seed	
routers/admin, lab	Corpus, user, competition
frontend/pages/*	Chat, Laws, Contracts, Compliance, Admin, Lab

Chi tiết kiểm thử số liệu và các vòng hỏi đáp em chạy tay nằm ở Chương 4 — chương này chỉ chốt thiết kế và quyết định hiện thực.

Chương 4. THỬ NGHIỆM VÀ ĐÁNH GIÁ

Chương này ghi lại những gì em đo và chạy trên máy làm đồ án. Em không đưa bảng F2 của cuộc thi vào đây như kết quả sản phẩm — lý do nói rõ ở cuối chương.

4.1 Môi trường thử nghiệm

Bảng 4.1: Cấu hình máy và cụm khi chạy thử.

Thành phần	Giá trị
Host	WSL2, Linux
Điều phối	Docker Compose, mạng cầu legal
PostgreSQL	17-alpine, cổng host 23432
Backend	Python 3.10, FastAPI, cổng 8023
Frontend	nginx phục vụ bản Vite, cổng 5173
LLM	gemini-2.5-flash, temperature 0
Embedding	gemini-embedding-001, 1536 chiều
Corpus	38 văn bản, 3.844 Điều

Một phần thử nghiệm cố ý chạy **không** gắn khóa API để chứng minh suy giảm có kiểm soát. Phần hỏi đáp có trích dẫn và soát xét hợp đồng cần khóa; khi hết hạn mức phải ra lỗi rõ, không stack trace.

4.2 Kiểm thử tự động

4.2.1 Tổ chức bộ test

Tổng 77 test: 53 backend (pytest) + 24 frontend (vitest). Backend không chạy trên SQLite. Fixture tạo database legal_assistant_test, chạy migration Alembic, hết phiên thì xóa. Em làm vậy vì FTS, unaccent và ràng buộc unique (công ty, rule, kỳ) không giả được bằng mock cho ra cảm giác “đã test”.

4.2.2 Các case viết vì từng vồ tay

- Hai tài khoản: hội thoại của A, B gọi GET bằng id đó — phải 404 (không 403).
- Đăng ký công ty khai quý / khai tháng — tập task tuân thủ khác nhau.
- Tìm “thue gtgt” không dấu — có hit, strategy full_text.
- Tải .txt — HTTP 201, status ready.

- Backend không có index vector — chat stream 503, /health vẫn 200.
- Hook SSE: server chỉ gửi token không gửi result — không được treo vĩnh viễn.

Frontend test không bật trình duyệt thật; chúng kiểm vòng parse sự kiện. Test xanh không chứng minh giao diện đẹp — chỉ chứng minh parser không gãy.

4.3 Chạy tay trên cụm Docker

Bảng 4.2: Kết quả smoke test trên cụm Docker em ghi lại.

Nhóm	Kết quả	Ý nghĩa
pytest	53 passed	Schema, FTS, cách ly, upload
vitest	24 passed	Parser SSE chịu thiếu event
GET /laws	38 văn bản	Catalog khớp manifest
Tìm “thue gtgt”	73 hit	Bỏ dấu hoạt động
Đăng ký DN 12 người, GTGT quý	39 task	Có ATVSLĐ, không VAT tháng
Hỏi đáp không khóa / chưa index	HTTP 503	Suy giảm có kiểm soát
Tải hd.txt	201, ready	Trích chữ lúc tải

Ba lệnh tối thiểu: copy .env ở thư mục gốc (không phải backend/.env), docker compose up -d --build, nạp corpus bằng load_postgres.py.

4.4 Kịch bản hỏi đáp chạy tay

4.4.1 Câu trong phạm vi corpus

Câu demo: “Công ty mười hai người, nhân viên kinh doanh, thử việc được phép bao lâu?”

- BM25 bắt cụm “thử việc” trong Bộ luật Lao động.
- Vector kéo thêm vài Điều về hợp đồng xác định thời hạn vì gần nghĩa.
- llm_filter thường bỏ Điều về hợp đồng mùa vụ.
- Câu trả lời nêu Điều 25: thời gian thử việc theo loại công việc, không phải một con số 60 ngày cho mọi vị trí.
- Citation bấm ra đúng Điều trong kho.

4.4.2 Câu không phải pháp luật

“Hôm nay ăn gì.” Nút analyze_intent trả SKIP, không tốn truy hồi. Nếu không có nút này, pipeline đôi khi kéo Điều về an toàn thực phẩm — dễ làm người xem tưởng hệ thống loạn.

4.4.3 Câu ngoài corpus

“Thủ tục ly hôn khi vợ chồng đều ở nước ngoài.” Kho không có luật hôn nhân gia đình. Đầu ra phải nói không đủ căn cứ. Nếu mô hình vẫn viết đoạn khuyên chung, danh sách citation sau hậu kiểm phải rỗng.

4.5 Đánh giá module tra cứu và lịch tuân thủ

Bảng 4.3: Đối chiếu hồ sơ doanh nghiệp và task được sinh.

Hồ sơ	Quan sát	Kết luận
12 NLĐ, GTGT quý	39 task; có ATVSLĐ; không VAT tháng	Lọc <code>applies_to</code> đúng
3 NLĐ, GTGT tháng	Có VAT tháng; không ATVSLĐ	Ngưỡng 10 người hoạt động
Đổi quý → tháng rồi sinh lại	Task cũ đã tick không mất	Unique (công ty, rule, kỳ)

Ô tìm không dấu “thue gtgt” trả 73 dòng; chữ trên trang vẫn còn dấu nhờ bôi đậm phía client. Đây là chỗ em coi là đạt so với mục tiêu YC-05.

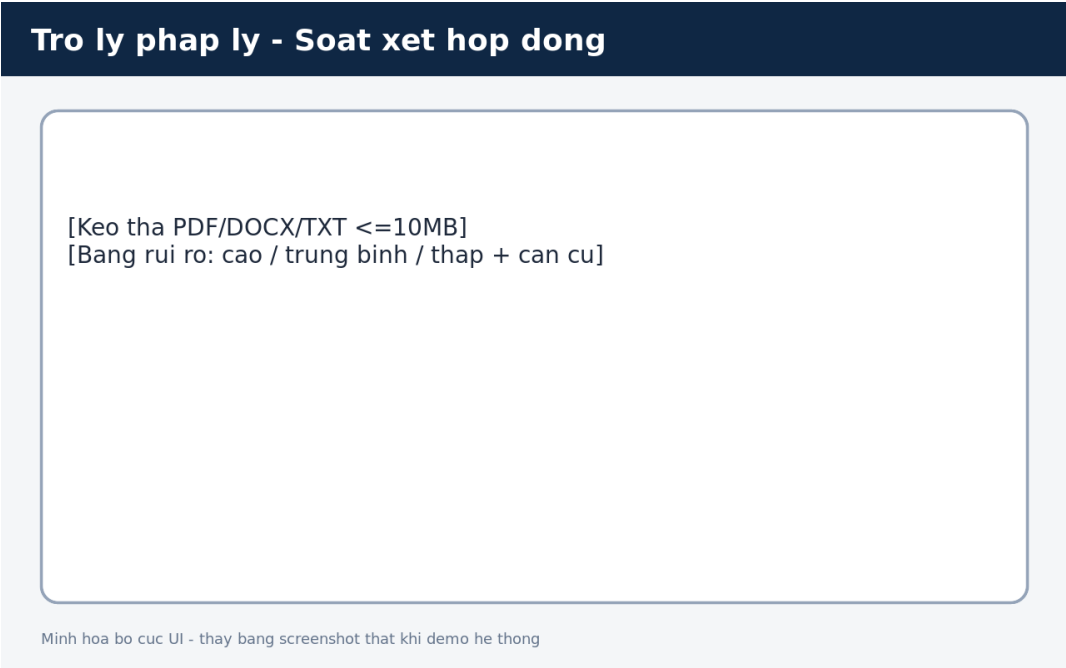
4.6 Đánh giá giao diện (định tính)



Hình 4.1: Bố cục trang hỏi đáp: cột hội thoại, vết pipeline, câu trả lời và citation.



Hình 4.2: Bố cục trang tra cứu: cây Chương–Điều và tìm không dấu.



Hình 4.3: Bố cục trang hợp đồng: upload và bảng rủi ro.

Tro ly pháp ly - Lịch tuân thủ

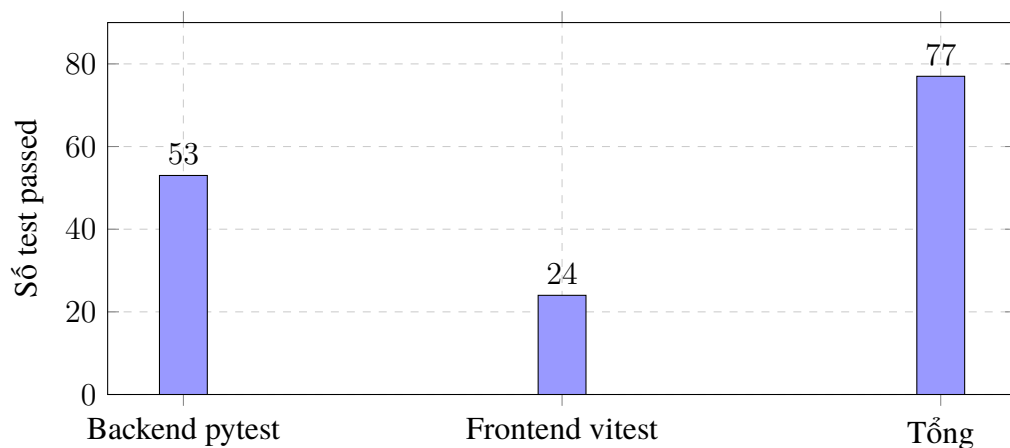
[Tổng | Qua hạn | 30 ngày | Đã xong]
[Nhóm theo tháng; legal_refs mô Điều gốc]

Mình họa bố cục UI - thay bằng screenshot thật khi demo hệ thống

Hình 4.4: Bố cục trang lịch tuân thủ: ô đếm và nhóm theo tháng.

Em tự dùng và nhờ bạn học không chuyên luật bấm thử. Phản hồi chung: cột vết pipeline giúp chờ đỡ sốt; nút “Hỏi AI về Điều này” trên trang luật được dùng nhiều; trên màn hẹp thanh hội thoại còn chật — ghi vào hạn chế.

4.7 Biểu đồ tóm tắt bộ kiểm thử



Hình 4.5: Phân bố số test tự động đạt trên máy làm đồ án.

4.8 Vì sao không có cột F2 cuộc thi trong báo cáo

Pipeline truy hồi gốc sinh trong cuộc thi hỏi đáp pháp lý, chấm F2 trên bộ câu và corpus khóa. Sản phẩm đồ án đổi embedding (Gemini, 1536 chiều), đổi LLM (Gemini Flash), tắt reranker

cục bộ, bật HyDE. Đổi một trong các thứ đó là đổi bài toán. Gắn số F2 bài thi vào chương này như kết quả đề án là không trung thực.

Lab /api/v1/lab/competition vẫn nhận JSON và trả đúng khuôn cũ để người có máy cục bộ Qwen tự chạy. Nếu làm tiếp: cùng corpus, cùng bộ vài trăm câu nội bộ, hai cột Qwen và Gemini, một bảng P/R/F2 — lúc đó mới bàn “đổi nhà cung cấp thì truy hồi thay đổi thế nào”.

4.9 Đối chiếu mục tiêu Chương 1

Bảng 4.4: Đối chiếu bảy mục tiêu cụ thể với bằng chứng.

MT	Nội dung rút gọn	Bằng chứng
1	Corpus cắt theo Điều	3.844 bản ghi, manifest 38 luật
2	Pipeline 7 bước + SSE	Code LangGraph + test SSE
3	Tra cứu cây + không dấu	73 hit “thue gtgt”
4	Soát xét hợp đồng nền	Upload 201, poll status
5	Lịch 12 rule	39 task với hồ sơ mẫu
6	JWT + cách ly	Test 404 chéo hội thoại
7	Docker + test	Compose 3 dịch vụ, 77 test

Mục tiêu em không nhận (F2 leaderboard mô hình mở) vẫn không nhận ở đây.

Chương 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Những gì đề án thực sự làm được

Em đã có một trang web chạy được trên Docker, nhiều tài khoản, nhiều công ty. Người dùng hỏi tiếng Việt thường, nhận câu có danh sách Điều bắt ra nguyên văn. Họ đọc được kho 38 văn bản theo cây, tìm được cả khi gõ thiếu dấu. Họ tải hợp đồng và nhận bảng rủi ro (khi có khóa mô hình). Họ có lịch các mốc thuế – bảo hiểm – lao động – BCTC sinh từ đúng hồ sơ mình khai.

Về kỹ thuật, chỗ em cho là đáng kể không nằm ở việc “dùng RAG” — hướng đó người khác làm nhiều. Chỗ đáng kể hơn là các quyết định nhỏ khi cài: cắt theo Điều; gộp BM25 với vector; gõ căn cứ không nằm trong tập truy hồi; tìm không dấu mà vẫn hiện có dấu; cột seq vì now() trùng nhau; 404 khi xem lên hội thoại; backend không tự sát khi thiếu API key.

Kiểm thử: 77 test, backend chạy trên Postgres thật. Em coi đó là phần bắt buộc của đề án ngành, không phải phụ lục cho có.

5.2 Những gì cố tình không làm, và những gì còn yếu

Không OCR — PDF ảnh fail lúc tải. Không hàng đợi Redis. Không chọn tự động bản luật còn hiệu lực khi hai số hiệu trùng tên khác năm. Không so F2 Gemini với F2 Qwen trên cùng bộ câu. Giao diện chưa tối cho điện thoại. Token đang để localStorage.

Hậu kiểm chỉ bắt số Điều, không bắt được chuyện mô hình diễn giải sai Điều đúng. HyDE và lọc LLM tốn hạn mức; trên gói miễn phí, nhúng lại 3.844 Điều để bị 429 — vì vậy mới có chia lô và nghỉ dần.

5.3 Hướng phát triển

1. OCR trước khi trích chữ, hoặc từ chối rõ “đây là PDF ảnh”.
2. Cột hiệu lực (ngày ban hành, hết hiệu lực) trên từng Điều, lọc khi truy hồi.
3. Đo F2 trên bộ vài trăm câu nội bộ, cùng corpus, hai endpoint mô hình.
4. Đưa soát xét hợp đồng sang worker riêng khi file dài.
5. Viện dẫn chéo thành đồ thị cho câu hỏi đa văn bản.
6. Cookie HttpOnly + CSP; đưa token ra khỏi localStorage.
7. Bố cục hẹp cho màn điện thoại — kế toán hay hỏi lúc đang ở cửa hàng.

5.4 Kết luận

Đồ án bắt đầu từ chuyện một công ty nhỏ hỏi luật trên mạng và không biết câu nào bịa. Em không giải được chuyện đó cho mọi ngành luật. Em giải, ở mức một đồ án ngành, cho một kho hẹp và bốn việc lặp lại của DNNVV.

Nếu mỗi lần hỏi, người dùng đều mở được Điều gốc ra đọc trước khi copy câu máy vào tờ khai hay vào hợp đồng, thì sản phẩm đã làm đúng việc nó nhận.

TÀI LIỆU THAM KHẢO

- [1] Dương Hữu Thành, *Công nghệ phần mềm*, NXB Thông tin và Truyền thông, 2024.
- [2] H. T. Duong, T. P. Bui, D. H. Nguyen, “QLoRA and RAG Application in Building Admissions Support Chatbot: Performance Evaluation and Knowledge Update Ability,” in N. N. Dao, V. Truong Hoang, F. Dornaika (eds.), *Explainable Intelligence in Digital Twins*, EIDT 2025, LNEE vol. 1531, Springer, Singapore, 2026.
- [3] P. Lewis, E. Perez, A. Piktus *et al.*, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” *NeurIPS*, vol. 33, pp. 9459–9474, 2020.
- [4] V. Karpukhin, B. Oğuz, S. Min *et al.*, “Dense Passage Retrieval for Open-Domain Question Answering,” *Proc. EMNLP*, pp. 6769–6781, 2020.
- [5] S. Robertson, H. Zaragoza, “The Probabilistic Relevance Framework: BM25 and Beyond,” *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.
- [6] G. V. Cormack, C. L. A. Clarke, S. Buettcher, “Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods,” *Proc. SIGIR*, pp. 758–759, 2009.
- [7] L. Gao, X. Ma, J. Lin, J. Callan, “Precise Zero-Shot Dense Retrieval without Relevance Labels,” *Proc. ACL*, pp. 1762–1777, 2023.
- [8] Quốc hội, *Luật Hỗ trợ doanh nghiệp nhỏ và vừa*, số 04/2017/QH14.
- [9] Quốc hội, *Bộ luật Lao động*, số 45/2019/QH14.
- [10] Quốc hội, *Luật Quản lý thuế*, số 38/2019/QH14.
- [11] PostgreSQL Global Development Group, “Full Text Search,” tài liệu trực tuyến, <https://www.postgresql.org/docs/current/textsearch.html>.

PHỤ LỤC

Phụ lục A. Chạy hệ thống trên máy

Sao `.env.example` thành `.env` ở thư mục gốc dự án, không phải trong `backend/`. Compose không đọc `backend/.env`. Điền `LLM_API_KEY` (trang AI Studio của Google) và `JWT_SECRET_KEY` (ví dụ `openssl rand -hex 32`). Chạy:

```
docker compose up -d --build
```

Lần đầu nạp luật:

```
docker compose exec backend python scripts/load_postgres.py--truncate
```

Rồi restart backend. Vào `http://localhost:5173`. Tài khoản đăng ký đầu tiên thành quản trị. Tra cứu dùng được ngay. Hỏi đáp cần đã nhúng vector: vào `/admin/corpus` bấm `reindex`, hoặc đợi startup nếu khóa đã có từ đầu.

Thiếu khóa: web vẫn mở, hỏi đáp trả 503 chữ tiếng Việt, không phải stack trace.

Phụ lục B. Chạy test

Backend (cần Postgres đang mở cổng 23432):

```
cd backend && uv run pytest
```

Frontend:

```
cd frontend && npm test
```

Tổng 77 test. Database test tên `legal_assistant_test`, tạo và xóa trong phiên `pytest`, không dùng dữ liệu demo.

Phụ lục C. Một số đường API

Đường	Việc
POST <code>/api/v1/auth/register</code>	Tạo user, có thể kèm organization
POST <code>/api/v1/auth/login</code>	Cấp access + refresh
POST <code>/api/v1/auth/refresh</code>	Đổi access, kiểm tra type
GET <code>/api/v1/conversations</code>	Danh sách hội thoại của đúng user
POST <code>/api/v1/legal/chat/stream</code>	SSE hỏi đáp
GET <code>/api/v1/laws</code>	Catalog
GET <code>/api/v1/laws/search</code>	FTS
GET <code>/api/v1/laws/{id}/articles</code>	Cây Chương–Điều
POST <code>/api/v1/documents</code>	Tải file
POST <code>/api/v1/documents/{id}/review</code>	202, chạy nền

Đường	Việc
GET /api/v1/compliance/tasks	Lịch
POST /api/v1/admin/corpus/reindex	Nhúng lại, chỉ admin
POST /api/v1/lab/competition	Bộ câu hỏi, chỉ admin

Phụ lục D. Lab thi đấu

POST /api/v1/lab/competition nhận JSON danh sách câu, trả file kết quả đúng khuôn cũ. Dừng khi cần số liệu với mô hình mở. Sản phẩm mặc định trên Docker không bật competition mode cho user thường.

Phụ lục E. Ghi chú biên dịch báo cáo

File `main.tex` phải chạy XeLaTeX (gói `fontspec`). Trong Cursor hoặc VS Code, biên dịch bằng `recipe xelatex` (hoặc `latexmk -xelatex`). Đầu file có chỉ thị TEX program trở sang xelatex. Ảnh minh họa đang để chế độ demo của `graphicx`; khi có file trong thư mục `Baocao/images/` thì bỏ tùy chọn demo trên lệnh nạp gói.